interruption terminate the thread, so you can just let the exception propagate up. But if you let exceptions propagate out of the thread function passed to the `std::thread` constructor, `std::terminate()` will be called, and the whole program will be terminated. In order to avoid having to remember to put a `catch (thread_interrupted)` handler in every function you pass to `interruptible_thread`, you can instead put that `catch` block inside the wrapper you use for initializing the `interrupt_flag`. This makes it safe to allow the interruption exception to propagate unhandled, because it will then terminate just that individual thread. The initialization of the thread in the `interruptible_thread` constructor now looks like this:

```
internal_thread=std::thread([f,&p]{
        p.set_value(&this_thread_interrupt_flag);
        try
        {
            f();
        }
        catch(thread_interrupted const&)
        {}
    });
```

Let’s now look at a concrete example where interruption is useful.

9.2.7. Interrupting background tasks on application exit

Consider for a moment a desktop search application. As well as interacting with the user, the application needs to monitor the state of the filesystem, identifying any changes and updating its index. Such processing is typically left to a background thread, in order to avoid affecting the responsiveness of the GUI. This background thread needs to run for the entire lifetime of the application; it will be started as part of the application initialization and left to run until the application is shut down. For such an application this is typically only when the machine itself is being shut down, because the application needs to run the whole time in order to maintain an up-to-date index. In any case, when the application is being shut down, you need to close down the background threads in an orderly manner; one way to do this is by interrupting them.